



Bank Campaign Call Optimization

Machine Learning for Smarter Telemarketing

Team 10 – Doritos



01 Business Understanding

The Problem

Our bank runs outbound telemarketing campaigns to sell term deposits. Calling broadly wastes time and budget because most customers say “no”.

Our Objective

Business Question: Which customers should we call first, today, to maximize conversions and reduce cost per acquisition (CPA)?

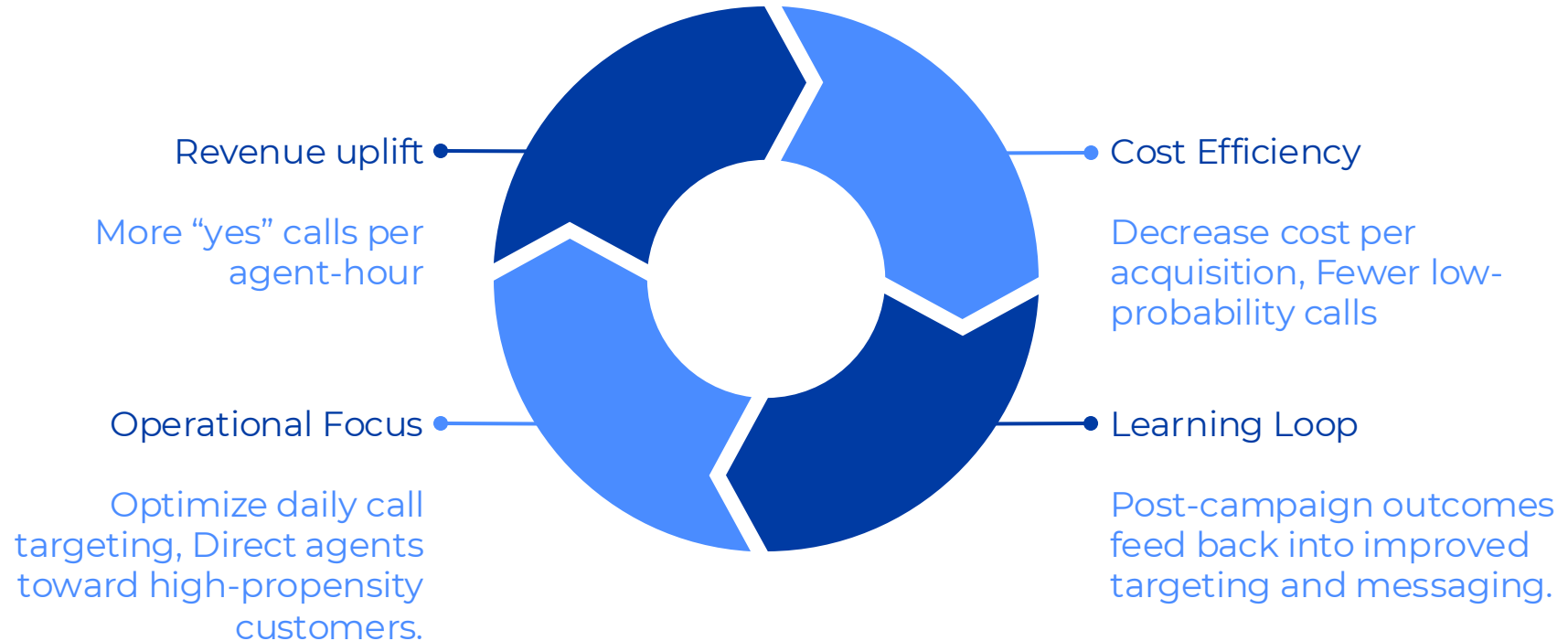
Build a supervised classification model that predicts, for each contactable customer, the probability of subscribing to a term deposit.

Stakeholders

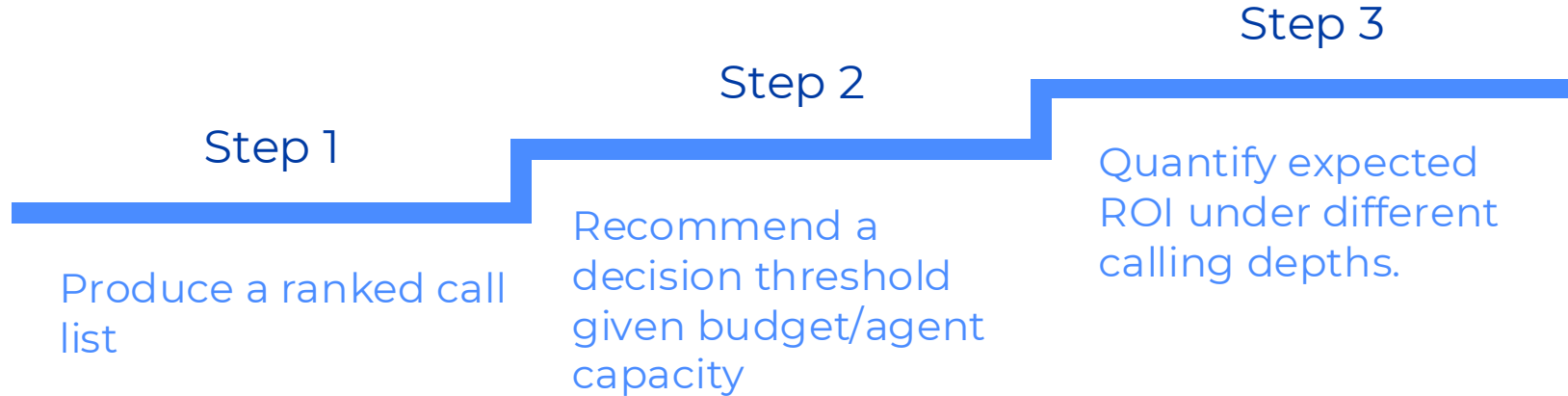
Marketing Leadership: How many top-ranked customers do we call this week? What budget delivers the best marginal ROI?

Call-center operations: How to schedule agents and set daily calling depth?

Why this matters?



Business Roadmap





Scope

One customer contact instance = one row.
Target is whether the customer subscribes ("yes"/"no").

Constraints

Data is historical (2008–2010); today's macro environment may differ. Some features (e.g., call duration) can leak information and must be handled carefully.

Assumptions

Similar campaign mechanics, comparable customer mix, and stable mapping from features → outcome in the near term.

Risk and Mitigation

Data Leakage

Exclude or carefully scenario-test features not known pre-call (notably *duration*).

Class Imbalance

Use metrics and resampling/thresholding strategies suited for rare positives.

Model Drift

Monitor performance and retrain periodically as campaigns or customer behavior change.



02 Data

Understanding

Data Sources and Units of Analysis

We use the Bank Marketing dataset (UCI). Each row is a single customer contact made during a telemarketing campaign. The target variable y indicates whether the customer subscribed to a term deposit (“yes”/“no”).

Core dataset (“bank-full”) ~45k rows, 16 input variables capturing customer profile, prior campaign history, and details of the last contact.

Augmented dataset (“bank-additional-full”) ~41k rows, 20 input variables including five macro/market indicators (e.g., euribor3m, emp.var.rate, nr.employed), which can add predictive signal.

We proceed with the augmented dataset for stronger signal and run a sensitivity analysis on the core dataset to confirm robustness.

Data Loading & Structure

- No parsing errors or missing column headers were detected.
- The dataset did not contain any NaN values. However, several categorical attributes used "unknown" as a placeholder for missing or undisclosed information. These were found mainly in: job, education, default, housing, loan
- Rather than imputing or removing these values, they were retained as their own distinct category, as prior research and inspection suggested that customers who withhold financial information often exhibit distinct behavior.
- Each categorical variable was visualized using count plots and target rate bar charts (showing the proportion of “yes” responses per category).
- Ordinal categories (like education) were plotted in logical order (e.g., illiterate, basic.4y, ..., university.degree, unknown).

Variable Description and Feature Groups

Variable	Description	Type	Example Value
age	Age of the client (numeric)	int64	56
job	Type of job (categorical)	object	housemaid
marital	Marital status (categorical)	object	married
education	Level of education (ordinal categorical)	object	basic.4y
default	Has credit in default? (categorical: yes, no, unknown)	object	no
housing	Has housing loan? (categorical: yes, no, unknown)	object	no
loan	Has personal loan? (categorical: yes, no, unknown)	object	no
contact	Communication type (telephone or cellular)	object	telephone
month	Last contact month of year	object	may
day_of_week	Last contact day of the week	object	mon
duration	Last contact duration in seconds (numeric — leakage variable)	int64	261
campaign	Number of contacts during this campaign (numeric)	int64	1
pdays	Days since last contact from a previous campaign (999 = not previously contacted)	int64	999
previous	Number of contacts before this campaign (numeric)	int64	0
poutcome	Outcome of the previous marketing campaign	object	nonexistent
emp_var	Employment variation rate — quarterly indicator	float64	1.100000
price_idx	Consumer price index — monthly indicator	float64	93.994000
conf_idx	Consumer confidence index — monthly indicator	float64	-36.400000
euribor_rate	Euribor 3-month interest rate — daily indicator	float64	4.857000
num_employed	Number of employed persons — quarterly indicator	float64	5191.000000
y	Target variable: has the client subscribed to a term deposit? (yes/no)	object	no

Customer profile: age, job, marital, education, default, housing, loan.

Current campaign contact details: contact (cellular/telephone), month, day_of_week/day, duration*.

Campaign intensity/history: campaign (contacts in current campaign), pdays (days since last prior contact), previous, poutcome (prior campaign outcome).

Macro context (augmented dataset): emp.var.rate, cons.price.idx, cons.conf.idx, euribor3m, nr.employed.

Target: y (yes/no).

Planned Exploratory Analysis

Target prevalence by segment

Subscription rate by contact channel (cell vs telephone), by month and day_of_week (seasonality/week effects), by poutcome and previous (prior success is a strong signal) and by campaign depth (diminishing returns of repeated contacts)

Macro alignment (augmented dataset)

Subscription rate vs. euribor3m and emp.var.rate (economic headwinds/tailwinds)

Correlation/association heatmaps

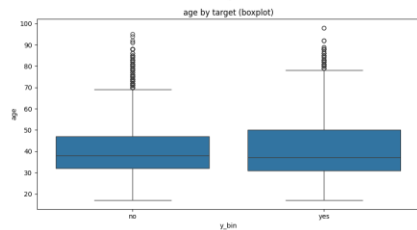
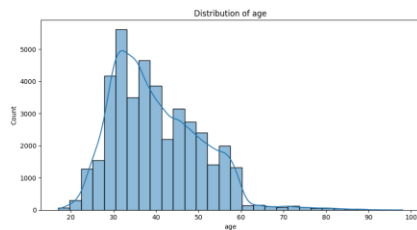
Cramér's V (categorical-categorical), point-biserial (numeric-binary target)

Lift and gains curves

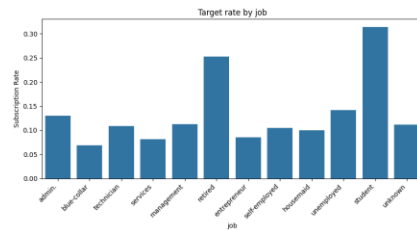
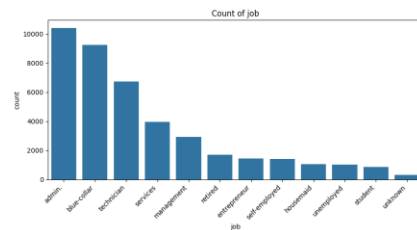
Early preview of how concentrated positives are in the top deciles—useful for shaping the eventual call-depth recommendation.

Demographic Data

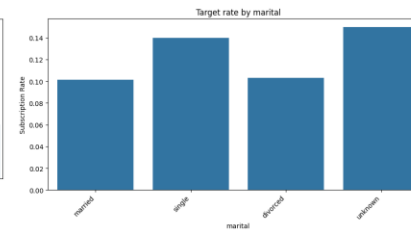
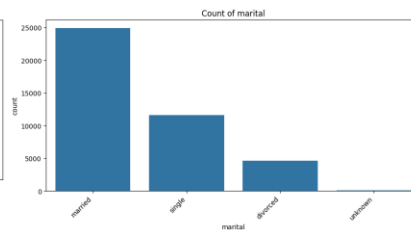
Age



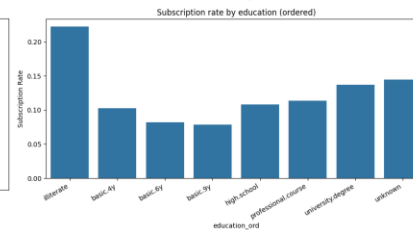
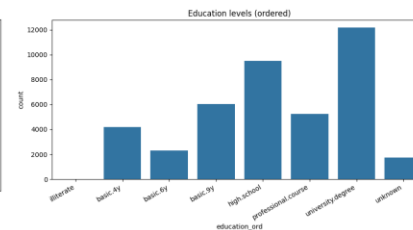
Job



Marital

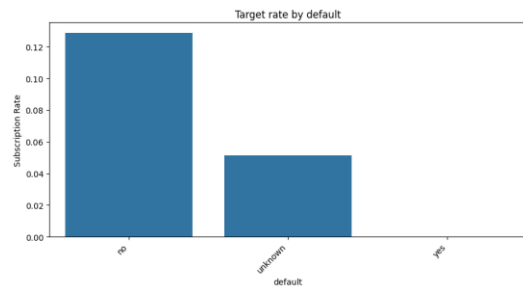
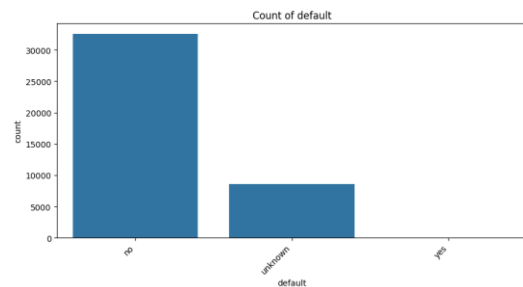


Education

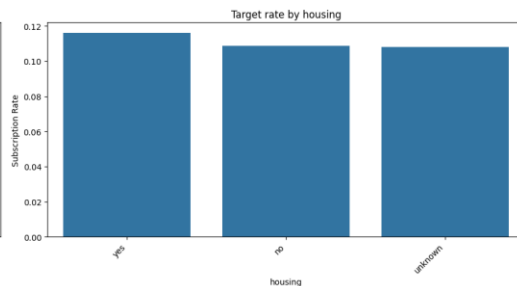
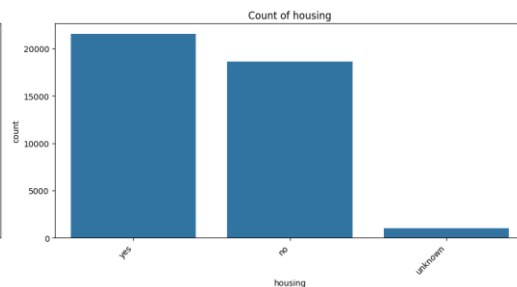


Finance Data

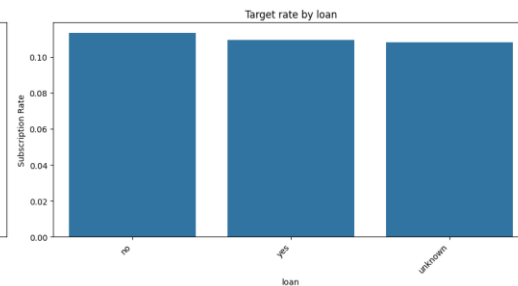
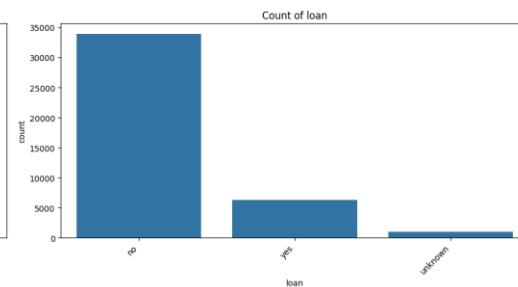
Default



Housing

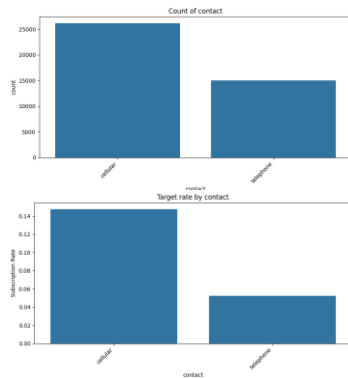


Loan

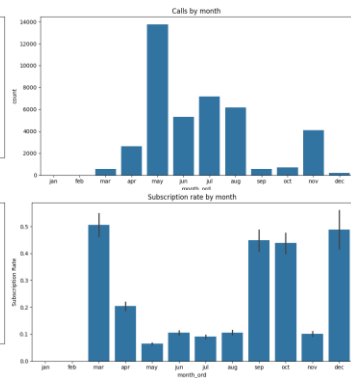


Campaign Data

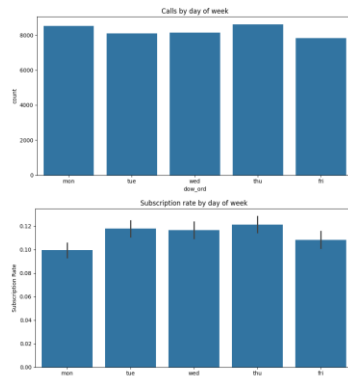
Contact



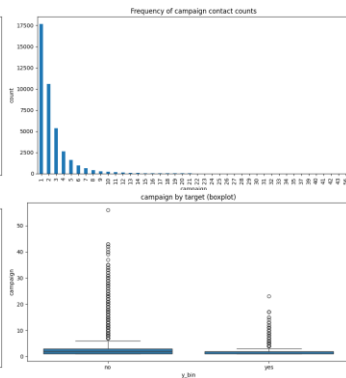
Month



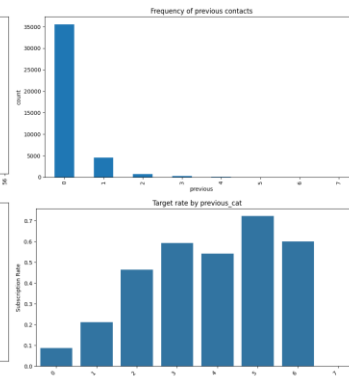
Day of the week



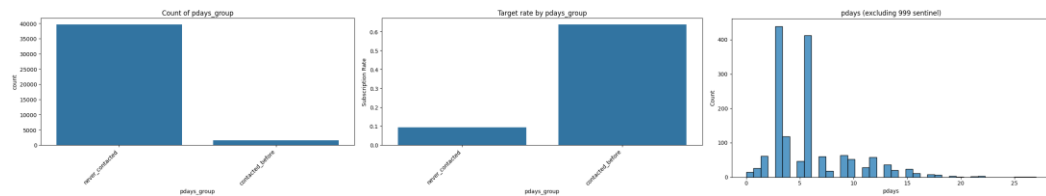
Campaign



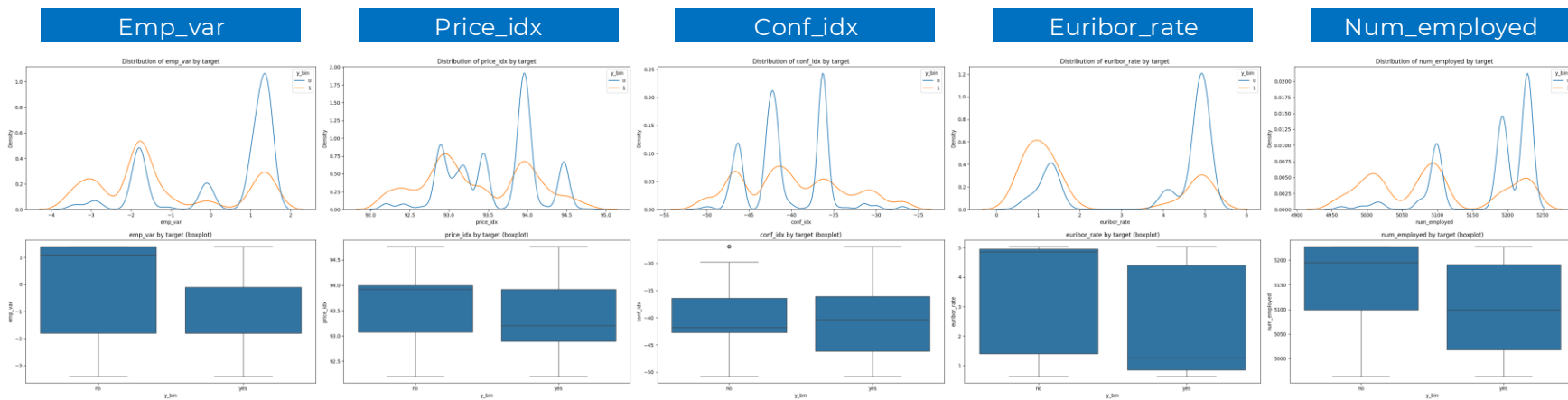
Previous Contacts



Pdays



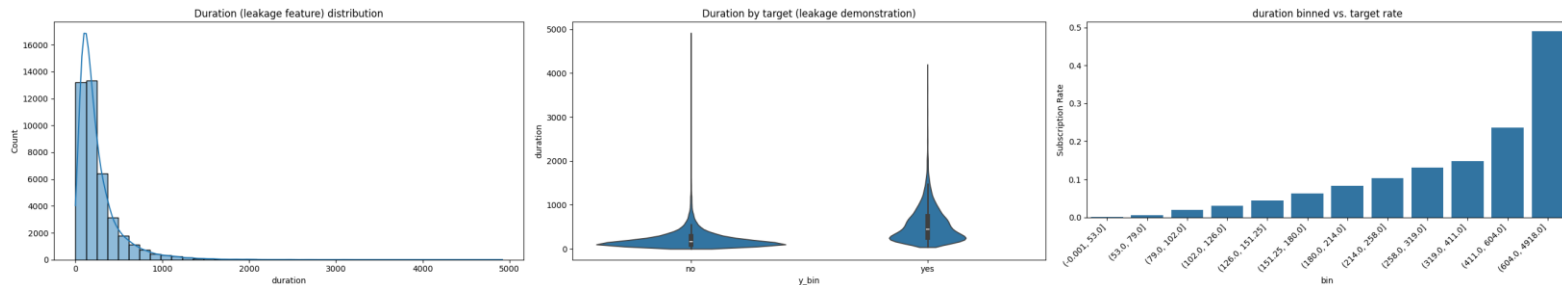
Macroeconomic Data



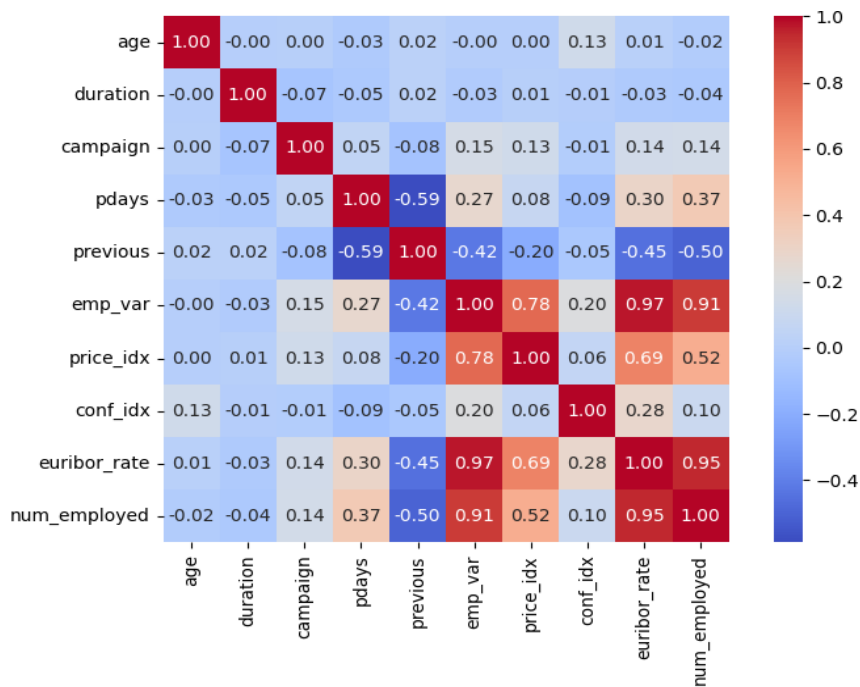
Pairplots were generated for macroeconomic variables to visualize their joint distributions and separation by target.

Duration Variable

- The duration variable (last contact length in seconds) was identified as leakage, since it is only known after the call is completed and is almost perfectly predictive of the outcome.
- This variable was excluded from all modeling and feature correlation analyses but was still visualized during EDA to demonstrate its strong relationship with the target variable.



Correlation Heatmap



Correlation heatmap for numeric variables revealed:

- High multicollinearity among macroeconomic indicators (emp_var, euribor_rate, and num_employed).
- Minimal correlation between demographic features and campaign variables.

Outlier and Variance Checks

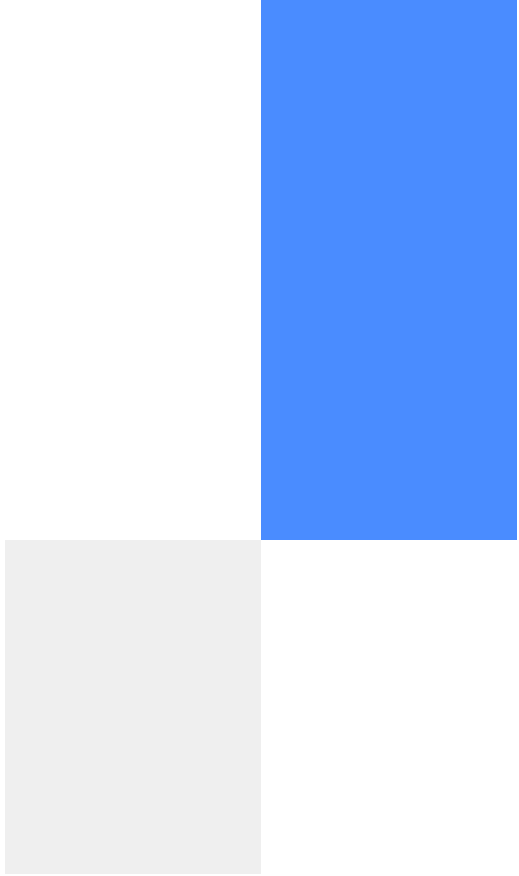
Boxplots were used to examine outliers.

- campaign and previous contained long tails (a few clients with very high contact counts).
- age and economic variables had minimal outliers.

Outliers were not removed since they represented real marketing scenarios rather than erroneous data.

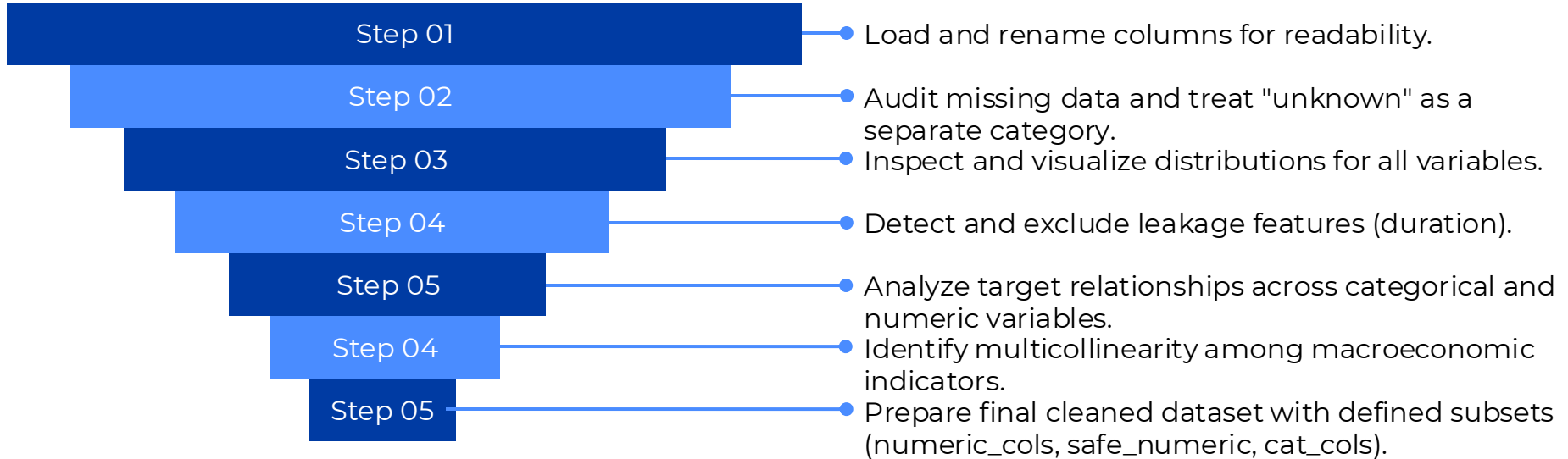
Outlier shares (IQR rule):

	outlier_share
previous	13.66%
duration	7.19%
campaign	5.84%
pdays	3.68%
age	1.14%
conf_idx	1.09%
emp_var	0.00%
price_idx	0.00%
euribor_rate	0.00%
num_employed	0.00%



03 Data Preparation

Visual Overview of Data Preparation Workflow



Addressing High Multicollinearity (VIF)

What we did

- Excluded known macro co-movers: euribor_rate, emp_var.
- Removed the leakage variable duration (post-call only, not usable pre-call).
- Built a numeric-only frame, dropped NA rows and non-varying columns.
- Removed variables with low prediction power to tackle overfitting

Why it matters

High multicollinearity inflates coefficient variance → unstable odds ratios and noisy interpretations in logistic regression.

VIF (numeric, excluding duration):

	feature	VIF
7	euribor_rate	64.342881
4	emp_var	33.070180
8	num_employed	31.662087
5	price_idx	6.337933
6	conf_idx	2.648960
3	previous	1.795642
2	pdays	1.612602
1	campaign	1.033689
0	age	1.018878

Feature Engineering & Data Split

What we did

- Target setup & leakage control
- Encoding strategy
- Scaling
- Three pre-processing variants
- Train–test split (hold-out)

Why it matters

Avoids data leakage, encodes categories correctly, stabilizes model estimation, and ensures the train/test distributions match—so offline performance is trustworthy and deployable.

Data Integrity and Readiness Summary

After cleaning and analysis, the prepared dataset included:

- 41,188 valid observations
- 10 input features (5 categorical, 5 numeric)
- No missing values
- Simplified macroeconomic variable names
- Exclusion of the leaky duration variable
- Identification of key correlations and outlier behavior

Target Variable Encoding

- The target column `y` contained categorical values "yes" and "no".
- A binary variable `y_bin` was created, with: 1 = customer subscribed to a term deposit
- 0 = customer did not subscribe

Exploratory Target Insights

EDA visualizations demonstrated key behavioral insights:

- Longer call durations almost always resulted in successful conversions.
- Most “yes” responses came from clients contacted fewer times (campaign ≤ 3).
- Clients not contacted for a long time (pdays = 999) tended to respond negatively.
- Economic downturns (low emp_var and high euribor_rate) corresponded with reduced success rates.



04

Modeling

Cross Validation and Generalization Performance

- Classification supervised learning algorithm
- We built Logistic Regression, Decision Tree and k-NN models to determine which would be the best performing one
- The decision tree model was the performing model prior to hyperparameter tuning

	accuracy	precision	recall	f1	roc_auc
k-NN	0.839305	0.368672	0.596972	0.455707	0.781984
Logistic Regression	0.826333	0.346974	0.612670	0.442883	0.786042
Decision Tree	0.833131	0.352326	0.572028	0.435599	0.747916

Hyperparameter Tuning

Standardize numeric & ordinal features for all three models

Conducted additional preparation for the Logistic Regression model

- Removed the top 2 features with highest VIF - euribor_rate and emp_var

Used nested cross-validation with hyperparameter search

- 5 folds each in inner and loops
- Inner loop – For parameter optimization
- Outer loop – for understanding generalization performance

Model Results

	Logistic Regression	k-NN	Decision Tree
Hyper- parameter Grid Search	C = 1, Penalty = l2, solver: lbfgs	n_neighbors = 203 weights = uniform p = 1	Criterion = gini max_depth = 10 min_samples_split = 4



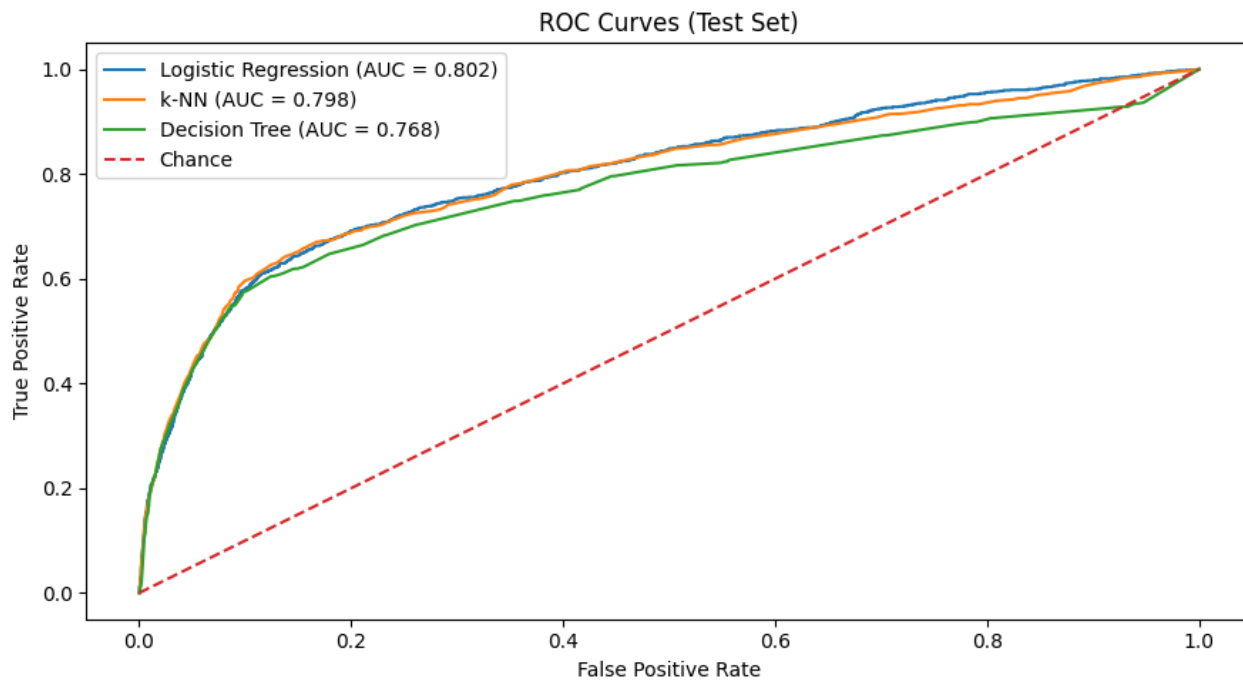
05

Evaluation

Model Performance

	Logisitic Regression	k-NN	Decision Tree
Accuracy	0.830	0.828	0.828
Precision	0.359	0.356	0.351
Recall	0.643	0.651	0.619
F1- SCORE	0.461	0.460	0.448
AUC-ROC	0.802	0.798	0.768

ROC Curve

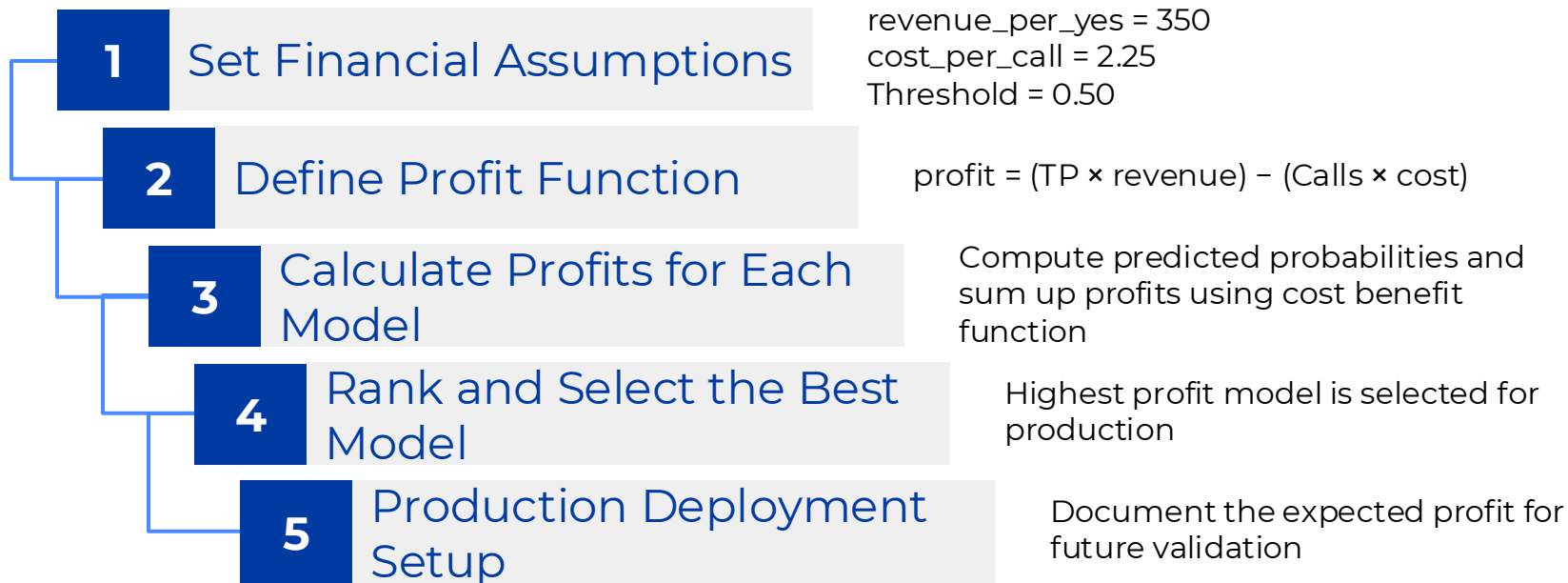




06

Deployment

Setup



Business Implication

Profit for Random Classifier
Profit = \$46k +- 4k

Profit for our Model
Expected = \$310k



07

Appendix

Creating Cost-Benefit Matrices

```
revenue_per_yes = 350.0
cost_per_call   = 2.25
THR = 0.50

def profit_at_threshold(y_true, proba, thr, revenue_per_yes, cost_per_call):
    pred = (proba >= thr).astype(int)
    tp = int(((pred == 1) & (y_true == 1)).sum())
    fp = int(((pred == 1) & (y_true == 0)).sum())
    calls = tp + fp
    return tp * revenue_per_yes - calls * cost_per_call

leaderboard = []
probas_cache = {}

for name, mdl in fitted_models.items():
    p = mdl.predict_proba(X_test)[: , 1]
    probas_cache[name] = p
    prof = profit_at_threshold(y_test, p, THR, revenue_per_yes, cost_per_call)
    leaderboard.append((name, prof))

leaderboard.sort(key=lambda x: x[1], reverse=True)
best_name, best_profit = leaderboard[0]

# store the production pick
prod_model = fitted_models[best_name]
chosen_threshold = THR # keep with the model for deployment

print("== Profit leaderboard (threshold =", THR, ") ==")
for n, prof in leaderboard:
    print(f"{n}: profit={prof:,.2f}")
print("\nSelected for prod:", best_name, "| Expected profit:", f"{best_profit:,.2f}")
```

Finding the best threshold for profit maximization

```
def save_model(model, path="final_model.joblib"):
    # Think of this like "saving your progress in a video game"
    # It stores your trained model into a file (so you don't lose it).
    dump(model, path)
    print(f"Saved model to {path}")

def load_model(path="final_model.joblib"):
    # This is like "loading your saved game"
    # It reopens the file so you can use the trained model again (for predictions).
    model = load(path)
    print(f"Loaded model from {path}")
    return model
```

Python

Deploying on New Data

🔗 Generate + Code + Markdown

```
def score_batch(model, X_new, id_col=None):

    # Get predicted probability of 'yes'
    scores = model.predict_proba(X_new)[: , 1]

    # Build a simple dataframe with the scores
    results = pd.DataFrame({"score_yes": scores})

    return results
```

Python

Scoring New Data

 Generate  Code  Markdown

```
scored = score_batch(prod_model, X_test, id_col=None)
scored.head()
```

Python

```
scored_sorted = scored.sort_values("score_yes", ascending=False).reset_index(drop=True)
scored_sorted.head()
```

Python

```
THRESHOLD = 0.65
call_list_thr = scored_sorted[scored_sorted["score_yes"] >= THRESHOLD].copy()
call_list_thr["rank"] = call_list_thr["score_yes"].rank(method="first", ascending=False).astype(int)
```

Python

```
call_list_thr.shape[0]
```

Python

Exporting the Call List

```
call_list_thr.to_csv("call_list_threshold.csv", index=False)
```

Python

```
import numpy as np

# business values
revenue_per_yes = 350.0
cost_per_call = 2.25

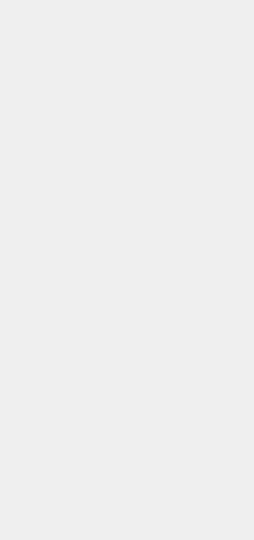
# choose what fraction of customers you'd randomly call
call_rate = 0.1 # e.g., call 10% of all customers

def random_classifier_profit(y_true, revenue_per_yes, cost_per_call, call_rate, n_runs=1000):
    n = len(y_true)
    profits = []
    for _ in range(n_runs):
        calls = np.random.rand(n) < call_rate
        tp = np.sum(calls & (y_true == 1))
        fp = np.sum(calls & (y_true == 0))
        calls_total = tp + fp
        profit = tp * revenue_per_yes - calls_total * cost_per_call
        profits.append(profit)
    return np.mean(profits), np.std(profits)

# Example usage
mean_profit, std_profit = random_classifier_profit(y_test, revenue_per_yes, cost_per_call, call_rate=0.1)

print(f"Random classifier expected profit (calling {call_rate:.0%}): ${mean_profit:,.2f} ± {std_profit:,.2f}")
```

Python



Thank You